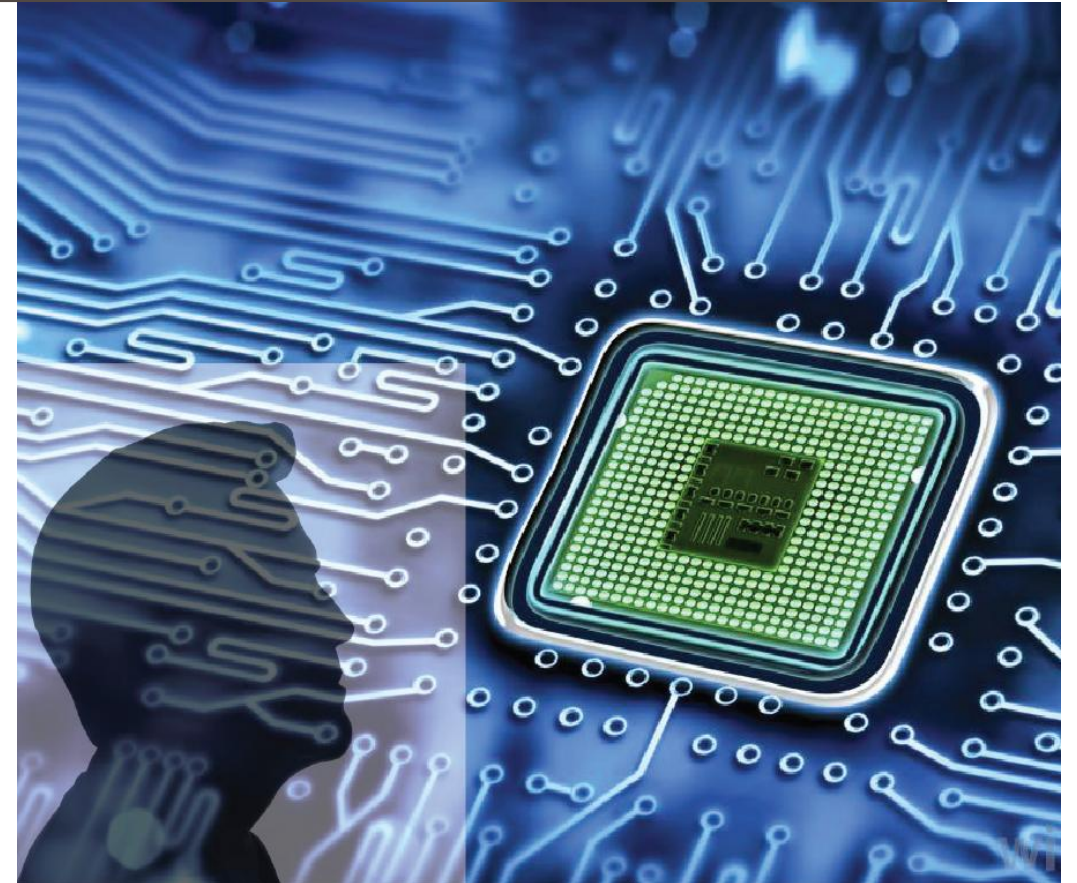


ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-10 Lecture-1 SystemVerilog

Sneh Saurabh
30th October, 2025



SystemVerilog



Randomization

Random Variables: Cyclic Random Variables

- **rand** variables: each time new value or repeated value can come
- **randc** variable: does not repeat before every possible value is assigned

```
ncsim> run
Course code: 510 Grade: 9
Course code: 514 Grade: 8
Course code: 513 Grade: 8
Course code: 515 Grade: 9
Course code: 512 Grade: 8
Course code: 511 Grade: 8
```

```
class Generator;
    randc integer course_code; // declare cyclic random number
    rand integer grade; // declare random number
    constraint c1 { // define constraints
        course_code >= 510;
        course_code <= 515;
        grade >= 6;
        grade <= 10;
    }
    function void display;
        $display("Course code: %0d Grade: %0d", course_code, grade);
    endfunction
endclass

program MyRand();
    initial begin
        Generator gen;
        gen = new(); // create object with random data
        for (int i=0; i <6; i++) begin
            gen.randomize(); // generate random values
            gen.display();
        end
    end
endprogram
```

Random Variables: Turn off

- We can choose a subset of variable to randomize
 - Pass the variables as argument to the `randomize` function
- We can turn off randomness of a random variable
 - Using the function `rand_mode(0)` with the variable

```
class Generator;
  randc integer course_code; // declare cyclic random number
  rand integer grade; // declare random number
  constraint c1 { // define constraints
    course_code >= 510;
    course_code <= 515;
    grade >= 0;
    grade <= 10;
  }
  function void display;
    $display("Course code: %0d Grade: %0d", course_code, grade);
  endfunction
endclass
```

```
program MyRand();
  initial begin
    Generator gen;
    gen = new(); // create object with random data
    gen.randomize(); gen.display();
    gen.randomize(); gen.display();
    gen.randomize(grade); // grade will only randomize, course_code same
    gen.display();
    gen.grade.rand_mode(0); // grade will not randomize, only course_code randomize
    gen.randomize(); gen.display();
  end
endprogram
```

```
Course code: 510 Grade: 4
Course code: 514 Grade: 0
Course code: 514 Grade: 3
Course code: 513 Grade: 3
```

Randomization: Status

- Randomization can fail if there are conflicting constraints
 - **randomize** function returns 0 for failure
 - We should check this status and give meaningful message

```
class Generator;
  randc integer course_code; // declare cyclic random number
  rand integer grade; // declare random number
  constraint c1 { // define constraints
    course_code >= 510;
    course_code <= 515;
    grade >= 10;
    grade <= 0;
  }
  function void display;
    $display("Course code: %0d Grade: %0d", course_code, grade);
  endfunction
endclass
```

```
program MyRand();
  initial begin
    Generator gen;
    gen = new(); // create object with random data
    if (gen.randomize())
      $display("Randomize successful");
    else
      $display("Randomize FAIL");
    gen.c1.constraint_mode(0);
    gen.randomize(); gen.display();
  end
endprogram
```

```
ncsim> run
          if (gen.randomize())
          |
ncsim: *W,SVRNDF (./random-4.sv,19|18): The randomize method call failed. The unique id o
f the failed randomize call is 0.
Observed simulation time : 0 FS + 1
ncsim: *W,RNDOCS: These constraints contribute to the set of conflicting constraints:

          grade >= 10; (./random-4.sv,7)
          grade <= 0; (./random-4.sv,8)
ncsim: *W,RNDOCS: These variables contribute to the set of conflicting constraints:

rand variables:
          grade [./random-4.sv, 3]

Randomize FAIL
Course code: 393217815 Grade: 1220233827
Simulation complete via implicit call to $finish(1) at time 0 FS + 1
(./random-4.sv,15:ncsim: MyRand())
```

Randomization: Only Checking Constraints

- To check that whether the constraints are satisfied for the current value of random numbers we can use `randomize(null)`
 - returns 0 if any constraint violated

```
class Generator;
  randc integer course_code; // declare cyclic random number
  constraint c1 { // define constraints
    course_code >= 510;
    course_code <= 515;
  }
  function void display;
    $display("Course code: %0d", course_code);
  endfunction
endclass
```

```
program MyRand();
  initial begin
    Generator gen;
    gen = new(); // create object with random data
    gen.randomize(); gen.display();
    gen.course_code = 317;
    gen.display();
    if (!gen.randomize(null))
      $display("Constraint violated");
    else
      $display("Constraint valid");
  end
endprogram
```

```
ncsim> run
Course code: 510
Course code: 317
      if (!gen.randomize(null))
      |
ncsim: *W,SVRNDF (./random-5.sv,19|19): The randomize method call failed. The unique id o
f the failed randomize call is 1.
Observed simulation time : 0 FS + 1
ncsim: *W,RNDOCS: These constraints contribute to the set of conflicting constraints:

      course_code >= 510; (./random-5.sv,4)
ncsim: *W,RNDOCS: These variables contribute to the set of conflicting constraints:

state variables:
      course_code (317) [./random-5.sv, 2] randc rand_mode(0)

Constraint violated
```

Constraints: Operators

- Constraints are declarative code defined within curly braces {}
 - In contrast to procedural code within begin-end
 - All code of the constraint is active at the same time
- Constraints can use relational operators $<$, $<=$, $=$, $>=$, $>$ (use only one per line)
- Equivalence operator $==$
 - Do not use $=$ for giving a fixed value to a variable
- Set membership: **inside** operator

```
program MyRand();
  initial begin
    Generator gen;
    gen = new(); // create object with random data
    for (int i=0; i<10; i++) begin
      gen.randomize(); gen.display();
    end
  end
endprogram
```

```
class Generator;
  const bit [31:0] CONGEST_ADDR = 42;
  rand bit test_congestion;
  rand bit [31:0] len, src;
  constraint c1 { // define constraints
    if (test_congestion) {
      src == CONGEST_ADDR;
    } else {
      src inside {0, [2:10], [100:107]};
    }
    len > 0;
    len < 1000;
  }
  function void display;
    $display("test_congestion: %b len: %0d src: %0d", test_congestion, len, src);
  endfunction
endclass
```

```
ncsim> run
test_congestion: 0 len: 386 src: 8
test_congestion: 0 len: 445 src: 10
test_congestion: 0 len: 369 src: 4
test_congestion: 0 len: 892 src: 107
test_congestion: 0 len: 128 src: 7
test_congestion: 0 len: 883 src: 107
test_congestion: 1 len: 898 src: 42
test_congestion: 0 len: 906 src: 106
test_congestion: 0 len: 418 src: 105
test_congestion: 0 len: 133 src: 104
```


Constraints: Bidirectional Behavior

- Constraints are solved bidirectionally
 - Constraints on all random variables are solved concurrently
 - Adding or removing constraint on any one variable affects all variables that are related directly or indirectly

```
class Bidir;  
  rand bit [15:0] r, s, t;  
  constraint c_bidir {           // All are solved in parallel  
    r < t;                       // A value for r affects s, t  
    s == r;  
    t < 10;  
    s > 5;  
  }  
endclass
```

- Even when there is no lower limit on t , it gets defined due to s and r

Table 6.1 Solutions for bidirectional constraint

<i>Solution</i>	<i>r</i>	<i>s</i>	<i>t</i>
A	6	6	7
B	6	6	8
C	6	6	9
D	7	7	8
E	7	7	9
F	8	8	9

Constraints: Guiding probabilities

```
class Generator;
  const bit [31:0] CONGEST_ADDR = 42;
  rand bit test_congestion;
  rand bit [31:0] len, src;
  constraint c1 { // define constraints
    if (test_congestion) {
      src == CONGEST_ADDR;
    } else {
      src inside {0, [2:10], [100:107]};
    }
    len > 0;
    len < 1000;
  }
  function void display;
    $display("test_congestion: %b len: %0d src: %0d", test_congestion, len, src);
  endfunction
endclass
```

```
ncsim> run
test_congestion: 0 len: 386 src: 8
test_congestion: 0 len: 445 src: 10
test_congestion: 0 len: 369 src: 4
test_congestion: 0 len: 892 src: 107
test_congestion: 0 len: 128 src: 7
test_congestion: 0 len: 883 src: 107
test_congestion: 1 len: 898 src: 42
test_congestion: 0 len: 906 src: 106
test_congestion: 0 len: 418 src: 105
test_congestion: 0 len: 133 src: 104
```

```
program MyRand();
  initial begin
    Generator gen;
    gen = new(); // create object with random data
    for (int i=0; i<10; i++) begin
      gen.randomize(); gen.display();
    end
  end
endprogram
```

- The probability distribution depends on the order of solving for variables

- When *src* is solved first, its value is picked from 19 possible values
 - Then *test_congestion* is assigned to meet the constraint

Constraints: Guiding probabilities using **solve ... before**

- We can use the **solve ... before** construct to change the probability distribution
 - It does not change the solution space

```
constraint c1 { // define constraints
  if (test_congestion) {
    src == CONGEST_ADDR;
  } else {
    src inside {0, [2:10], [100:107]};
  }
  len > 0;
  len < 1000;
  solve test_congestion before src;
}
```

- When *test_congestion* is solved first, its value is picked from two possible values
 - Then *src* is assigned to meet the constraint

```
ncsim> run
test_congestion: 1 len: 209 src: 42
test_congestion: 0 len: 945 src: 107
test_congestion: 1 len: 898 src: 42
test_congestion: 0 len: 688 src: 9
test_congestion: 1 len: 136 src: 42
test_congestion: 1 len: 510 src: 42
test_congestion: 0 len: 790 src: 102
test_congestion: 1 len: 628 src: 42
test_congestion: 1 len: 355 src: 42
test_congestion: 0 len: 273 src: 103
```

Constraints: Implication

- Implication operator: $\rightarrow$
- $A \rightarrow B$ means: When A is true B has to be true, when A is false B can be anything (true or false)

```
class Generator;
  rand bit x;
  rand bit [1:0]y;
  constraint c_xy { // define constraints
    (x==0) -> (y==0);
    solve x before y;
  }
  function void display;
    $display("x: %d y: %0d", x, y);
  endfunction
endclass
```

```
x: 0 y: 0
x: 0 y: 0
x: 1 y: 0
x: 1 y: 2
x: 0 y: 0
x: 0 y: 0
x: 0 y: 0
x: 1 y: 3
x: 0 y: 0
```

Table 6.2 Implication operator truth table

$A \rightarrow B$	$B=false$	$B=true$
$A=false$	true	true
$A=true$	false	true

```
program MyRand();
  initial begin
    Generator gen;
    gen = new(); // create object with random data
    for (int i=0; i<10; i++) begin
      gen.randomize(); gen.display();
    end
  end
endprogram
```

- Implication can be computed in either direction

```
class Generator;
  rand bit x;
  rand bit [1:0]y;
  constraint c_xy { // define constraints
    (x==0) -> (y==0);
    solve y before x;
  }
  function void display;
    $display("x: %d y: %0d", x, y);
  endfunction
endclass
```

```
ncsim> run
x: 1 y: 1
x: 0 y: 0
x: 1 y: 2
x: 1 y: 3
x: 1 y: 2
x: 1 y: 3
x: 1 y: 2
x: 1 y: 2
x: 1 y: 2
x: 1 y: 1
```

Table 6.5 Solutions for Impl class

Solution	x	y	Probability
A	0	0	1/2
B	0	1	0
C	0	2	0
D	0	3	0
E	1	0	1/8
F	1	1	1/8
G	1	2	1/8
H	1	3	1/8

- Equivalence operator: $\leftrightarrow$
- $A \leftrightarrow B$ means: $A \rightarrow B$ and $B \rightarrow A$

Constraints: Weighted Distribution

- We can skew the distribution in a particular direction to accelerate finding bugs or cover cases
- We can change the distribution using **dist** keyword
- Two operators:
 - **:=** weight is the same for every value specified in the range
 - **:/** weight is equally divided among all the values specified in the range
- Weights can be constant or variable (allows changing weight on the fly)

```
class Transaction;  
  rand bit [1:0] src, dst;  
  constraint c_dist {  
    src dist {0:=40, [1:3]:=60};  
    // src = 0, weight = 40/220  
    // src = 1, weight = 60/220  
    // src = 2, weight = 60/220  
    // src = 3, weight = 60/220  
  
    dst dist {0:/40, [1:3]:/60};  
    // dst = 0, weight = 40/100  
    // dst = 1, weight = 20/100  
    // dst = 2, weight = 20/100  
    // dst = 3, weight = 20/100  
  }  
endclass
```

Constraints: Inline

- Sometimes we need to add constraints during randomization (easy to understand for simple constraints)
- We can use inline constraint using **randomize with**

- The inline constraint gets added to the existing (active) constraints
- Scope of inline constraint is the class

```
ncsim> run
data: 1003 address: 1
data: 8631 address: 16
```

```
class Generator;
  rand bit [0:15]data;
  rand bit [0:8]addr;
  constraint c { // define constraints
    data > 1000;
    addr < 24;
  }
  function void display;
    $display("data: %0d address: %0d", data, addr);
  endfunction
endclass

program MyRand();
  initial begin
    Generator gen;
    gen = new(); // create object with random data
    gen.randomize() with {data < 1024;};
    gen.display();
    gen.randomize() with {addr == 16;};
    gen.display();
  end
endprogram
```

Random Number Functions

- Instead of calling `randomize()` we can directly call built-in random number functions
- `randomize()` assigns values to random variables in a class and also meets constraint
 - This behavior is different from the built-in random number functions

- `$random`: flat distribution, returning signed 32-bit random
- `$urandom`: flat distribution, returning unsigned 32-bit random
- `$urandom_range`: number within a range

```
a = $urandom_range(3, 10); // Pick a value from 3 to 10
a = $urandom_range(10, 3); // Pick a value from 3 to 10
b = $urandom_range(5);     // Pick a value from 0 to 5
```

- `$dist_normal`: bell-shaped
- `$dist_poisson's`: bell-shaped
- `$dist_uniform`: flat

Randomization: `pre_randomize` and `post_randomize`

- Sometimes we need to call a function before/after randomization
 - Example: we want to set limit before randomization
 - Example: based on random data, we want to compute error correction bit
- SystemVerilog has automatically created `pre_randomize()` and `post_randomize()` functions for a class with random variables

```
class randc8;  
    randc bit [3:0] val;  
endclass
```

```
class UniqArray;  
    bit [3:0] ua[8];
```

```
function void pre_randomize();  
    randc8 rc8;  
    rc8 = new();  
    foreach (ua[i]) begin  
        rc8.randomize();  
        ua[i] = rc8.val;  
    end  
endfunction
```

```
function void display();  
    foreach (ua[i]) begin  
        $display("ua[%0d] = %0d", i, ua[i]);  
    end  
endfunction
```

```
endclass
```

```
program MyRand();  
    initial begin  
        UniqArray ua;  
        ua = new(); // create object with random data  
        ua.randomize();  
        ua.display();  
    end  
endprogram
```

```
ncsim> run  
ua[0] = 1  
ua[1] = 2  
ua[2] = 13  
ua[3] = 12  
ua[4] = 3  
ua[5] = 5  
ua[6] = 7  
ua[7] = 10
```


Randomizing Arrays of Data

- Number of items in an array can be randomized
- Values of individual items can be randomized

```
class RandomArray;  
  rand bit [0:7] arr[];  
  constraint c1 {  
    arr.size() > 0;  
    arr.size() <= 8;  
    foreach (arr[i]) arr[i] > 9;  
    foreach (arr[i]) arr[i] < 20;  
  }  
  function void display();  
    $write("size:%4d, values: ", arr.size());  
    foreach (arr[i]) $write("%4d ", arr[i]);  
    $display;  
  endfunction  
endclass
```

```
program MyRand();  
  initial begin  
    RandomArray ra;  
    ra = new(); // create object with random data  
    for (int i=0; i<6; i++) begin  
      ra.randomize();  
      ra.display();  
    end  
  end  
endprogram
```

```
ncsim> run  
size: 6, values: 13 10 10 13 18 17  
size: 3, values: 12 18 18  
size: 8, values: 17 14 12 13 15 17 12 15  
size: 7, values: 14 11 12 16 17 17 13  
size: 3, values: 15 18 14  
size: 2, values: 17 15
```

Randomizing Arrays of Handles

For an array containing handles to objects:

- **randomize()** on array invokes **randomize()** on each individual object that the handles point to
 - It will not randomize the handles themselves
 - Each object should be instantiated (object created using **new**) before calling **randomize()** on the array

```
program MyRand();  
  initial begin  
    RandomHandleArray ra;  
    ra = new(); // create object with random data  
    for (int i=0; i<6; i++) begin  
      ra.randomize();  
      ra.display();  
    end  
  end  
endprogram
```

```
class Data;  
  rand bit [3:0]data;  
endclass
```

```
class RandomHandleArray;  
  rand Data arr[];  
  constraint c1 {  
    arr.size()> 0;  
    arr.size()<=8;  
  }  
  function pre_randomize();  
    arr = new[10]; // more than the max size  
    foreach (arr[i]) arr[i] = new();  
  endfunction  
  function void display();  
    $write("size:%4d, values: ", arr.size());  
    foreach (arr[i]) $write("%4d ", arr[i].data);  
    $display;  
  endfunction  
endclass
```

```
ncsim> run  
size: 6, values: 7 3 12 10 3 7  
size: 2, values: 2 13  
size: 3, values: 6 12 6  
size: 3, values: 11 5 12  
size: 4, values: 13 6 12 11  
size: 5, values: 11 7 2 3 1
```

randcase

- Weighted choice between various actions can be made using **randcase ... endcase**
 - Does not need class and sometimes easy to use
 - Adds all the weights and then picks a branch based on its weight
- Adds all the weights and then picks a branch based on its weight

```
program MyRand();  
  initial begin  
    bit [1:0] len;  
    for (int i=0; i<10; i++) begin  
      randcase  
        1: begin len = 0; $display("Setting 0"); end  
        6: begin len = 1; $display("Setting 1"); end  
        2: begin len = 2; $display("Setting 2"); end  
        1: begin len = 3; $display("Setting 3"); end  
      endcase  
    end  
  end  
endprogram
```

```
ncsim> run  
Setting 0  
Setting 2  
Setting 1  
Setting 0  
Setting 1  
Setting 1  
Setting 2  
Setting 1  
Setting 3  
Setting 2
```

Randomization: Pseudo Random Number Generator (PRNG)

- Random numbers are generated using Pseudo Random Number Generator (PRNG)
 - PRNG is tool dependent
 - Different tools will produce different random number sequence
 - Same tool, different versions can produce different random number sequences

- Each object and thread has its own PRNG and unique seed
 - When a new object or thread is started, its PRNG is seeded from the parent's PRNG
 - If single seed started at simulation, creates several streams of random stimulus (each distinct)

- SystemVerilog has separate PRNG for each thread and object
 - This allows changing one object (example, adding one random variable in a class) having no impact on the objects of other classes
 - This is in contrast to Verilog that has a single PRNG for the entire simulation

Randomization: Impact of Seed

- Making change in code (creating another object) can lead to different random stimuli
- Because seed can change

```
class Rand1;  
  rand bit [3:0] val;  
  function void display();  
    $display("val: %0d", val);  
  endfunction  
endclass
```

```
class Rand2;  
  rand bit [3:0] val;  
endclass
```

```
program MyRand();  
  initial begin  
    Rand1 r1;  
    r1 = new(); // create object with random data  
    r1.randomize(); r1.display();  
    r1.randomize(); r1.display();  
    r1.randomize(); r1.display();  
  end  
endprogram
```

```
ncsim> run  
val: 13  
val: 1  
val: 10
```

```
program MyRand();  
  initial begin  
    Rand1 r1;  
    Rand2 r2;  
    r2 = new();  
    r1 = new(); // create object with random data  
    r1.randomize(); r1.display();  
    r1.randomize(); r1.display();  
    r1.randomize(); r1.display();  
  end  
endprogram
```

```
ncsim> run  
val: 5  
val: 9  
val: 1
```

- Effect can be reduced, if we make changes only after the existing ones

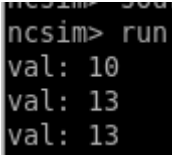
```
program MyRand();  
  initial begin  
    Rand1 r1;  
    Rand2 r2;  
    r1 = new(); // create object with random data  
    r1.randomize(); r1.display();  
    r1.randomize(); r1.display();  
    r1.randomize(); r1.display();  
    r2 = new();  
  end  
endprogram
```

```
val: 13  
val: 1  
val: 10
```

Randomization: Setting Seed

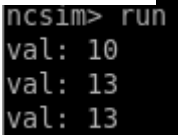
- Seed can be set using `process::srandom(seed)` within the thread and `obj.srandom(seed)` for the object

```
program MyRand();
  initial begin
    Rand1 r1;
    r1 = new(); // create object with random data
    r1.srandom(1234);
    r1.randomize(); r1.display();
    r1.randomize(); r1.display();
    r1.randomize(); r1.display();
  end
endprogram
```



```
ncsim> run
val: 10
val: 13
val: 13
```

```
program MyRand();
  initial begin
    Rand1 r1;
    Rand2 r2;
    r2 = new();
    r1 = new(); // create object with random data
    r1.srandom(1234);
    r1.randomize(); r1.display();
    r1.randomize(); r1.display();
    r1.randomize(); r1.display();
  end
endprogram
```



```
ncsim> run
val: 10
val: 13
val: 13
```

- Starting with the same seed, they produce the same sequence, even when a new object was created before the creation of this object

References

- Spear, Chris. SystemVerilog for Verification: A Guide to Learning the Testbench Language Features. Vol. 161. Springer, 2008.